STEP 1 · VIDEO ANALYSIS

A walk-in 3D store, not a web page

The clip is a first-person walkthrough of a stylised supermarket running in the browser. You steer with trackpad or mouse, glide down aisles, and click products. It is a real-time WebGL scene with a normal website layered on top.

3D layer (WebGL)

- First-person camera moving through aisles
- Low-poly, flat-lit shelves packed with product boxes
- Hanging blue aisle signs, such as Everyday Essentials and Home Care
- Numbered checkout lanes at the far end
- Shopping cart fixed at the bottom of the view

2D layer (HTML over canvas)

- Top bar: logo, search, language, cart button with item count
- Floating product card on hover or click: thumbnail, name, price, Add button
- Category labels that stay readable at any distance

Turn it into inventory or store management

Shopper mode

Walk aisles, tap a product, add to cart, check out at a lane. Search flies the camera to the shelf.

Manager mode

Same store, but each shelf slot shows stock. Click a product to edit price, restock, or reorder.

Live stock as visuals

Shelf fullness equals quantity. Low stock gets a red tag and a pulsing marker. Empty slots show a gap.

Admin extras

Floor-plan minimap, heatmap of top sellers, aisle editor, low-stock alerts, audit log.

Note: the video recreates a real retailer. Use your own brand, name and artwork.

STEP 2 · STACK, DESIGN SYSTEM, CLAUDE SETUP

Tech stack and design system

Front end

- Vite, React, TypeScript
- three.js with React Three Fiber and drei (PointerLockControls, Html, Instances, Environment)
- Zustand for cart, selection and mode state
- Tailwind CSS for the 2D overlay
- Optional: Rapier for collisions, Leva for tuning

Back end and ops

- Supabase: Postgres (products, shelves, stock), Auth, Realtime for live stock
- Stripe Checkout for payments
- Vercel hosting
- Playwright and Vitest for tests

3D assets

- Model shelves, signs and checkout in Blender, export GLB
- Compress with Draco or Meshopt, textures as KTX2
- Product boxes via InstancedMesh with one atlas texture
- Bake lighting; keep draw calls under about 200

Data model

- `products(id, sku, name, price, image)`
- `shelf_slots(id, aisle, x, y, z, product_id)`
- `stock(slot_id, qty, min_qty)`
- `orders, order_items, users`

Design system (original, not copied)

Colour

Deep blue #0B2A6F, signal red #E5322D, paper #F6F4EE, ink #14213D, shelf white #FFFFFF. Red is only for sale and low stock.

Type

Bricolage Grotesque for signs and headings, Inter for UI, JetBrains Mono for SKUs and numbers.

3D style

Flat-shaded low-poly, soft ambient light, one directional light, light fog for depth, rounded product boxes, no textures heavier than 1024 px.

UI

8 px grid, 12 px radius, 44 px touch targets, glass top bar, product card with 150 ms ease-out, WASD plus touch joystick, visible focus rings.

Claude setup to connect

Tool

Claude Code in your project folder. Put stack, design tokens and rules in CLAUDE.md.

Skills

web-3d-motion, frontend-design and modern-web-guidance for visuals and web APIs.

Connectors (MCP)

Context7 for current three.js and R3F docs. Three.js 3D Viewer to preview scenes. A Supabase MCP for the database. Canva or Exa for assets and research, if wanted.

Hooks

After edit: run tsc and eslint. Before tool use: block writes to .env. On stop: run tests and a build.

Subagents

scene-builder (models, lighting), inventory-api (Supabase, realtime), qa-perf (FPS, bundle size).

Build order

Empty room and movement, one aisle, instancing, product card, cart, database, manager mode, polish.

UPGRADE · FULL STACK FOR 3D, MOTION AND EFFECTS

Two build paths

Path A: inside Claude's website (Omega)

One HTML file. WebGL through three.js, plain JavaScript, HTML and CSS overlay, GSAP for motion, Web Audio for sound. Everything is generated in code. Node.js and Blender cannot run in the chat, so they are not used here.

Path B: professional build on your computer

Vite, React Three Fiber, drei, postprocessing, Node.js tooling and Blender models. Better visuals and a real data layer, but you need Node installed and an editor.

Stack by job

Rendering

WebGL 2 via three.js. ACES tone mapping, sRGB output, fog for depth, hemisphere plus one directional light, pixel ratio capped at 2.

Language and UI

JavaScript (TypeScript in Path B). HTML and CSS overlay with CSS variables, grid and flex, backdrop-filter glass bar, CSS transitions for buttons and drawers.

Motion

GSAP timelines for camera flights, doors and add-to-cart arcs. Damped lerp for look and move. Framer Motion for the React overlay in Path B.

Effects

Bloom on signs and ceiling lights, SSAO, vignette, soft shadows, floor reflection, dust particles with Points, depth of field when a product card opens. Path A fakes glow with emissive materials and sprites.

Node.js (local only)

Runs Vite, gltf-transform for compressing models, sharp for texture atlases, an optional Express mock API for products, Playwright for FPS and screenshot checks.

Blender (Path B)

Modular kit: shelf, sign, counter, cart, door. 1 unit = 1 m, UV unwrap, bake AO lightmaps, name objects like SHELF_Beauty_01, export GLB with Draco and KTX2. A Python script batch-exports.

Audio

Web Audio for generated footsteps, door whoosh

Quality tools

stats.js for FPS, Spector.js for draw calls, Lighthouse

STEP 3 · COPY AND PASTE

The master prompt

Give this to Claude Code after setting up CLAUDE.md. Ask for plan mode first, then build milestone by milestone.

```
Build a browser-based 3D store: "[YOUR STORE NAME]". A first-person walkthrough of a stylised supermarket with a 2D shopping UI over it, plus a manager mode for inventory.

STACK: Vite, React, TypeScript, three.js via React Three Fiber and drei, Zustand, Tailwind, Supabase (Postgres, Auth, Realtime), Stripe Checkout, Vercel.

SCENE: Low-poly flat-shaded store, 6 aisles, hanging blue category signs, shelves filled with instanced product boxes, 4 numbered checkout lanes, soft fog, one directional light. First-person camera: mouse look plus WASD on desktop, virtual joystick on mobile. Collisions with shelves.

UI OVERLAY: Top bar with logo, search, language switch, cart button with count. Clicking a product opens a floating card (image, name, price, Add). Search flies the camera to the product's shelf. Cart drawer and checkout at a lane.

MANAGER MODE (login required): Shelf slots show stock as fullness; low stock shows a red tag. Click a product to edit price, quantity and reorder level. Realtime stock updates. Minimap and low-stock list.

DATA: tables products, shelf_slots, stock, orders, order_items. Seed 60 sample products with placeholder images.

DESIGN: Deep blue #0B2A6F, red #E5322D (sale and low stock only), paper #F6F4EE. Fonts Bricolage Grotesque, Inter, JetBrains Mono. 8 px grid, 44 px touch targets, visible focus states.

PERFORMANCE: 60 FPS on a mid laptop, 30 FPS on phones. InstancedMesh for products, compressed GLB and KTX2, under 200 draw calls, lazy-load the 3D bundle.

PROCESS: Start in plan mode and list milestones. Use Context7 for current library docs. Run tsc, eslint and tests after each milestone. Do not use real brand names or logos.

DELIVER: running app, README with setup, .env.example, and a seed script.
```

Replace the bracketed name and add your products. Change one milestone at a time; asking for the whole store in one go gives weaker results.

OMEGA · ANALYSIS

What Omega must, should and could have

Scope: front end only, built by Claude on the website as one self-contained HTML page. No database, no real payment, no deployment. Everything is generated in code: shelves, products, signs and textures. That keeps it to one file with no downloads.

Must have

- First-person walking: drag to look, WASD or arrows, touch joystick on phones
- Store with entrance, 8 category aisles, hanging signs, back wall, checkout area
- Shelves filled with products (instanced meshes)
- Hover highlight, click for product card with name, price, Add
- Cart: count in top bar, drawer with quantity, remove, subtotal, tax, total
- Checkout counter with belt, cashier desk and terminal
- Pay flow: simulated card, cash or wallet, processing animation, receipt, reset
- Category jump and search that moves the camera to the shelf
- Collision so you cannot walk through shelves
- Loading screen, help overlay, mobile layout

Good to have

- Product flies into the cart when added
- 3D cart visible at the bottom of the view
- Sliding entrance doors
- Minimap with your position
- Sale tags and "only 3 left" badges
- Ceiling lights with soft glow, glossy floor
- Promo screen above the entrance
- List mode: a plain accessible product list as an alternative
- Quality toggle (Low, High) and reduced-motion respect
- Generated footstep and beep sounds, off by default

Nice extras

- Shoppers wandering in the aisles
- Pharmacy counter with a pharmacist
- Day and night lighting
- Wishlist and recently viewed
- Self-checkout kiosks

Aisles for Omega

- Beauty, Pharmacy, Food, Drinks
- Apparel, Electronics, Home, Kids
- Pharmacy items are generic, such as Pain Relief Tablets, with no health claims

How it should be built

Engine

three.js r128 from cdnjs, pinned, plus GSAP 3.12.5 for tweens. Write simple look and move controls yourself, because the core build has no pointer-lock add-on. Node.js and Blender are not used in this path; see the Stack board for the professional path.

Scale and layout

1 unit is 1 metre. Eye height 1.65. Store about 40 by 30 m, aisles 3 m apart, shelves 2.2 m tall, checkout near the entrance side.

Performance

One InstancedMesh per category, cheap Lambert materials, one hemisphere and one directional light, fake blob shadows, fog, pixel ratio capped at 2.

Product art

Draw boxes, bottles and bags with box geometry, and paint labels on a canvas texture atlas. No outside images.

Picking

Raycast on the centre or pointer; read instanceId to find the product. Hover scales that instance slightly.

State

One plain JS store for cart and mode. The 2D overlay is HTML over the canvas, not drawn in 3D.

Omega design system

OMEGA · COPY AND PASTE

Prompts for Claude on the website

Send Prompt 1 in a new chat. Then use 2 and 3 as follow-ups. One huge message tends to get cut off, so the build is split.

Prompt 1: build the store

```
Build "Omega", a realistic interactive 3D supermarket as ONE self-contained HTML file and publish it as an artifact. Front end only: no backend, no real payments.

TECH RULES: three.js r128 from https://cdnjs.cloudflare.com/ajax/libs/three.js/r128/three.min.js (pinned). Allowed extra library: GSAP 3.12.5 from https://cdnjs.cloudflare.com/ajax/libs/gsap/3.12.5/gsap.min.js for camera flights, doors and add-to-cart arcs. No other libraries, no external images, models or fonts except Google Fonts (Sora, Inter). Generate everything in code. Write your own first-person controls (drag to look, WASD and arrows to move, touch joystick on mobile). No localStorage.

STORE: 1 unit = 1 m, eye height 1.65, store about 40 x 30 m. Entrance with sliding doors, promo screen, 8 aisles with hanging signs: Beauty, Pharmacy, Food, Drinks, Apparel, Electronics, Home, Kids. Shelves 2.2 m tall filled with instanced products (boxes, bottles, bags) with painted label textures. Checkout area with 4 numbered counters, belts, terminals. Collision with shelves and walls. Bright, flat-shaded low-poly look, soft fog, hemisphere plus one directional light, no heavy shadows.

PRODUCTS: 12 products per aisle, invented brand names, realistic prices. Pharmacy items are generic (for example "Pain Relief Tablets 20 pack") with no health claims.

INTERACTION: hover highlights a product, click opens a floating card (name, category, price, quantity stepper, Add to cart). Add makes the product fly into a cart button in the top bar. Top bar: Omega logo, search, category menu, cart count. Search and category menu move the camera smoothly to that shelf.

CART AND PAYMENT: cart drawer with quantity, remove, subtotal, tax 8 percent, total. Button "Go to checkout" teleports the camera to a counter, where items appear on the belt. Payment screen is simulated: card, cash or wallet, fake card form, processing animation, receipt with order number, then "Continue shopping" resets the cart.

DESIGN: navy #101A3A, violet #5B3DF5, coral #FF5A4D, mint #19C39A, paper #F7F6F2. Sora for signs, Inter for UI. 12 px radius, 44 px touch targets, visible focus rings. UI motion 200 ms ease-out; respect prefers-reduced-motion.

EXTRAS: loading screen, help overlay with controls, minimap, accessible list-mode toggle.

PERFORMANCE: aim for 60 FPS, InstancedMesh per category, pixel ratio max 2, Lambert materials.

Build the complete file in one go. Keep the code organised in sections with comments.
```

Prompt 2: polish

```
Improve the Omega store: smoother camera movement with slight walking bob, glossy floor, glowing ceiling lights, 3D shopping cart visible at the bottom of the view, "only 3 left" and sale tags, a pharmacy counter, and a low and high quality toggle. Do not remove any existing feature. Output the full updated file.
```

Prompt 3: fix

```
Here is what is wrong: [DESCRIBE WHAT YOU SEE, e.g. "I walk through the Food shelves", "FPS is low on my phone", "the pay button does nothing"]. Fix only this, keep everything else, and tell me what you changed.
```